Complete Guide to Seaborn Plot Types

A comprehensive reference for Python statistical visualization

---

Overview

Seaborn is a Python data visualization library based on Matplotlib that provides a high-level interface for drawing attractive statistical graphics. Seaborn plots can be categorized into three main levels:

- **Figure-level plots** — Create entire figures with multiple subplots
- **Axis-level plots** — Create single plots on specific matplotlib axes
- **Functions by data type** — Distribution, categorical, relational, and matrix plots

Part 1: Figure-Level vs Axis-Level Plots

Figure-Level Plots

Figure-level functions create a complete figure with one or more subplots. They manage the entire figure automatically.

Key Characteristics:

- Return a **FacetGrid** object
- Cannot be placed on existing axes
- Support **col** and **row** parameters for faceting
- Include built-in legend management
- Automatically adjust figure size

Figure-Level Functions:

Function	Description	Parameters
sns.relplot()	Relational plots (scatter, line)	kind='scatter' or kind='line'
sns.displot()	Distribution plots (histogram, KDE, ECDF)	kind='hist', 'kde', or 'ecdf'
sns.catplot()	Categorical plots	kind='strip','swarm','box','violin','boxen','point','bar'
sns.lmplot()	Regression plots	Linear model plots with confidence bands
sns.pairplot()	Pairwise relationships	Matrix of scatterplots for multiple variables

Example:

```
import seaborn as sns
import matplotlib.pyplot as plt

# Figure-level plot with faceting
g = sns.relplot(data=tips, x='total_bill', y='tip',
                col='day', hue='smoker', kind='scatter')
plt.show()
```

Axis-Level Plots

Axis-level functions create a single plot on a specified matplotlib axes. They return the matplotlib axes object.

Key Characteristics:

- Return a matplotlib **Axes** object
- Can be placed on existing axes
- No automatic faceting (use FacetGrid directly for faceting)
- More control over customization
- Can be combined with other matplotlib plots

Axis-Level Functions:

Function	Description	Category
sns.scatterplot()	Scatter plot	Relational
sns.lineplot()	Line plot	Relational
sns.histplot()	Histogram	Distribution
sns.kdeplot()	Kernel Density Estimate	Distribution
sns.ecdfplot()	Empirical Cumulative Distribution	Distribution
sns.rugplot()	Rug plot for marginal distributions	Distribution
sns.stripplot()	Strip plot	Categorical
sns.swarmplot()	Swarm plot (avoid overlap)	Categorical
sns.boxplot()	Box plot	Categorical
sns.violinplot()	Violin plot	Categorical
sns.boxenplot()	Boxen/enhanced box plot	Categorical
sns.pointplot()	Point plot	Categorical
sns.barplot()	Bar plot	Categorical
sns.heatmap()	Heat map	Matrix
sns.clustermap()	Clustered heat map	Matrix
sns.regplot()	Regression plot	Regression
sns.residplot()	Residual plot	Regression

Example:

```
# Axis-level plot on specific axes
fig, ax = plt.subplots()
sns.scatterplot(data=tips, x='total_bill', y='tip',
                hue='smoker', ax=ax)
ax.set_title('Tips by Total Bill')
plt.show()
```

Part 2: Distribution Plots

Distribution plots visualize the distribution of one or more variables.

1. Histogram (histplot)

Displays the frequency distribution using bins.

```
sns.histplot(data=tips, x='total_bill', bins=30)
# Figure-level: sns.displot(data=tips, x='total_bill', kind='hist')
```

Parameters:

- **bins**: Number or sequence of bin edges
- **kde**: Whether to overlay KDE
- **stat**: 'count', 'frequency', 'density', 'probability'
- **multiple**: 'layer', 'dodge', 'stack', 'fill'

2. Kernel Density Estimate (kdeplot)

Smooth continuous probability density function.

```
sns.kdeplot(data=tips, x='total_bill', hue='day')
# Figure-level: sns.displot(data=tips, x='total_bill', kind='kde')
```

Parameters:

- **bw_method**: Bandwidth estimation method
- **cut**: Distance beyond extremes
- **shade**: Fill under curve
- **cumulative**: Cumulative distribution

3. Empirical Cumulative Distribution (ecdfplot)

Shows proportion of observations less than each value.

```
sns.ecdfplot(data=tips, x='total_bill')
# Figure-level: sns.displot(data=tips, x='total_bill', kind='ecdf')
```

Parameters:

- **stat**: 'proportion' or 'count'
- **complementary**: Show survival function (1-CDF)

4. Rug Plot (rugplot)

Plots tick marks for each data point on the axis.

```
sns.rugplot(data=tips, x='total_bill')
```

Parameters:

- **height**: Height of rug marks
- **expand_margins**: Whether to expand margins

5. Joint Distribution (jointplot)

Combines scatter and marginal distributions.

```
sns.jointplot(data=tips, x='total_bill', y='tip', kind='scatter')
# Options for kind: 'scatter', 'hist', 'hex', 'kde', 'reg'
```

Part 3: Categorical Plots

Categorical plots show relationships between categorical and numeric variables.

1. Strip Plot (stripplot)

Scatter plot with categorical axis.

```
sns.stripplot(data=tips, x='day', y='total_bill', jitter=True)
```

2. Swarm Plot (swarmplot)

Like stripplot but points are adjusted to avoid overlap.

```
sns.swarmplot(data=tips, x='day', y='total_bill')
```

3. Box Plot (boxplot)

Shows distribution through quartiles.

```
sns.boxplot(data=tips, x='day', y='total_bill')
```

Parameters:

- **whis**: Whisker length (IQR multiplier)
- **orient**: 'v' or 'h'
- **sym**: Symbol for outliers

4. Violin Plot (violinplot)

Combines box plot with KDE.

```
sns.violinplot(data=tips, x='day', y='total_bill')
```

Parameters:

- **inner**: 'box', 'quartile', 'point', 'stick'
- **split**: Split violins for hue

5. Boxen Plot (boxenplot)

Enhanced box plot for large datasets.

```
sns.boxenplot(data=tips, x='day', y='total_bill')
```

6. Point Plot (pointplot)

Plots point estimates with error bars.

```
sns.pointplot(data=tips, x='day', y='total_bill', capsize=0.2)
```

7. Bar Plot (barplot)

Shows aggregate statistics with error bars.

```
sns.barplot(data=tips, x='day', y='total_bill', estimator=np.mean)
```

8. Count Plot (countplot)

Counts categorical variable occurrences.

```
sns.countplot(data=tips, x='day')
```

Figure-level:

```
sns.catplot(data=tips, x='day', y='total_bill', kind='box')
```

Part 4: Relational Plots

Relational plots show relationships between multiple variables.

1. Scatter Plot (scatterplot)

Points plotted on two-dimensional space.

```
sns.scatterplot(data=tips, x='total_bill', y='tip', hue='day')
```

Parameters:

- **size**: Variable for point size
- **style**: Variable for marker style
- **alpha**: Point transparency

2. Line Plot (lineplot)

Lines connecting data points, suitable for time series.

```
sns.lineplot(data=flights, x='year', y='passengers')
```

Parameters:

- **marker**: Marker style
- **dashes**: Dash pattern
- **err_style**: 'band' or 'bars'

3. Relational Figure-Level (relplot)

Faceted scatter or line plots.

```
sns.relplot(data=tips, x='total_bill', y='tip',
            col='time', row='day', hue='smoker')
```

Part 5: Matrix Plots

Matrix plots visualize relationships in matrix form.

1. Heatmap (heatmap)

```
# Correlation matrix
corr = tips.corr()
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm')
```

Parameters:

- **annot**: Show values in cells
- **fmt**: Format string
- **cmap**: Color map
- **mask**: Mask certain cells
- **vmin**/**vmax**: Color scale limits

2. Clustermap (clustermap)

Heatmap with hierarchical clustering.

```
sns.clustermap(corr, annot=True, cmap='coolwarm')
```

Part 6: Regression Plots

Regression plots model relationships between variables.

1. Regression Plot (regplot)

```
sns.regplot(data=tips, x='total_bill', y='tip')
```

Parameters:

- **order**: Polynomial order
- **logistic**: Logistic regression
- **lowess**: Locally weighted regression

2. Residual Plot (residplot)

Plots residuals from a regression.

```
sns.residplot(data=tips, x='total_bill', y='tip')
```

3. LM Plot (lmplot)

Figure-level regression with faceting.

```
sns.lmplot(data=tips, x='total_bill', y='tip',
           col='day', hue='smoker')
```

Comparison Table: Figure-Level vs Axis-Level

Feature	Figure-Level	Axis-Level
Return type	FacetGrid	Axes
Can be placed on axes	No	Yes
Automatic faceting	Yes	No (use FacetGrid)
Figure management	Automatic	Manual
Multiple subplots	Easy	More complex
Customization	Limited	Extensive
Integration with matplotlib	Complex	Simple

Quick Reference: Choosing the Right Plot

Goal	Recommended Plot
Distribution of single variable	histplot(), kdeplot()
Distribution by category	boxplot(), violinplot()
Two variable relationship	scatterplot(), regplot()
Time series	lineplot()
Correlation matrix	heatmap()
Multiple variable relationships	pairplot()
Counts of categories	countplot()
Aggregate statistics	barplot(), pointplot()

Complete Example with Tips Dataset

```
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

# Load dataset
tips = sns.load_dataset('tips')

# 1. Figure-level relational plot
sns.relplot(data=tips, x='total_bill', y='tip',
            hue='day', col='time', kind='scatter')
plt.show()

# 2. Axis-level distribution plot
fig, ax = plt.subplots()
sns.histplot(data=tips, x='total_bill', kde=True, ax=ax)
ax.set_title('Distribution of Total Bill')
plt.show()

# 3. Categorical plot
sns.catplot(data=tips, x='day', y='total_bill',
            kind='violin', hue='sex', split=True)
plt.show()

# 4. Matrix plot
corr = tips.select_dtypes(include=[np.number]).corr()
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title('Correlation Matrix')
plt.show()

# 5. Pairwise relationships
sns.pairplot(data=tips, hue='sex')
plt.show()
```

Styling Options

Seaborn provides several built-in themes:

```
# Setting style
sns.set_style('whitegrid')      # Options: 'darkgrid', 'whitegrid', 'dark', 'white', 'ticks'
sns.set_context('paper')         # Options: 'paper', 'notebook', 'talk', 'poster'

# Color palettes
sns.set_palette('viridis')       # Color maps: 'deep', 'muted', 'bright', 'pastel', 'dark', 'colorblind'
sns.color_palette('husl', 8)     # Custom palette

# Removing spines (with whitegrid)
sns.despine(left=True, bottom=True)
```

Best Practices

1. Choose the right plot type for your data and question
2. Use figure-level plots for faceting and multi-plot layouts
3. Use axis-level plots for more control and integration with other plots
4. Add informative labels and titles
5. Use appropriate color palettes for colorblind accessibility
6. Consider data size — some plots (swarm, violin) are slow for large datasets
7. Adjust figure size for readability
8. Use seaborn.set_theme() for consistent styling
9. Document your parameters for reproducibility
10. Use interactive backends for exploration

This comprehensive guide covers all major Seaborn plot types organized by level and category. Use this as a reference when creating statistical visualizations in Python.